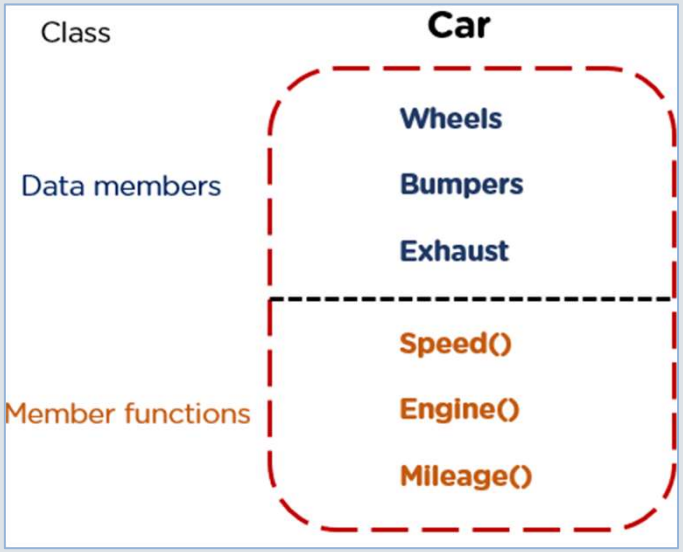
Object Oriented programing

Classes

A class is a data type whose variables are objects

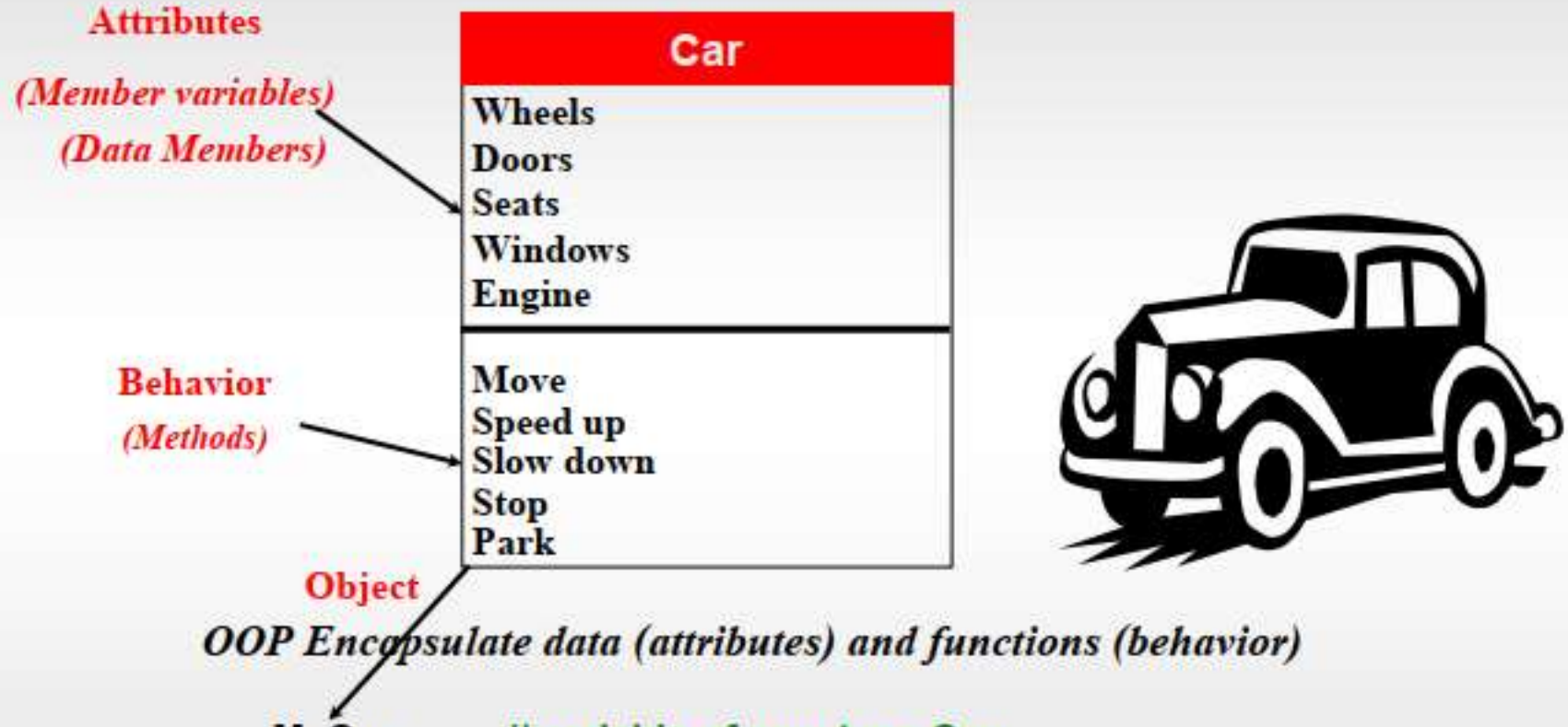
- **The definition of a class includes:**
  - **data member variables**
  - **member functions**



- **A class description is somewhat like a structure definition plus the member functions**
- **We call the data member *attributes.***

What is an Object?

- **Objects: Encapsulation**



- **The objects can be used to simulate real world entities.**
- **Promotes code reuse by allowing the programmer to specify a very general object (Person) and make it more specific (Student or Instructor) by allowing the programmer to build on their code.**

Class Circle

- **To create a new type named Circle as a class definition**

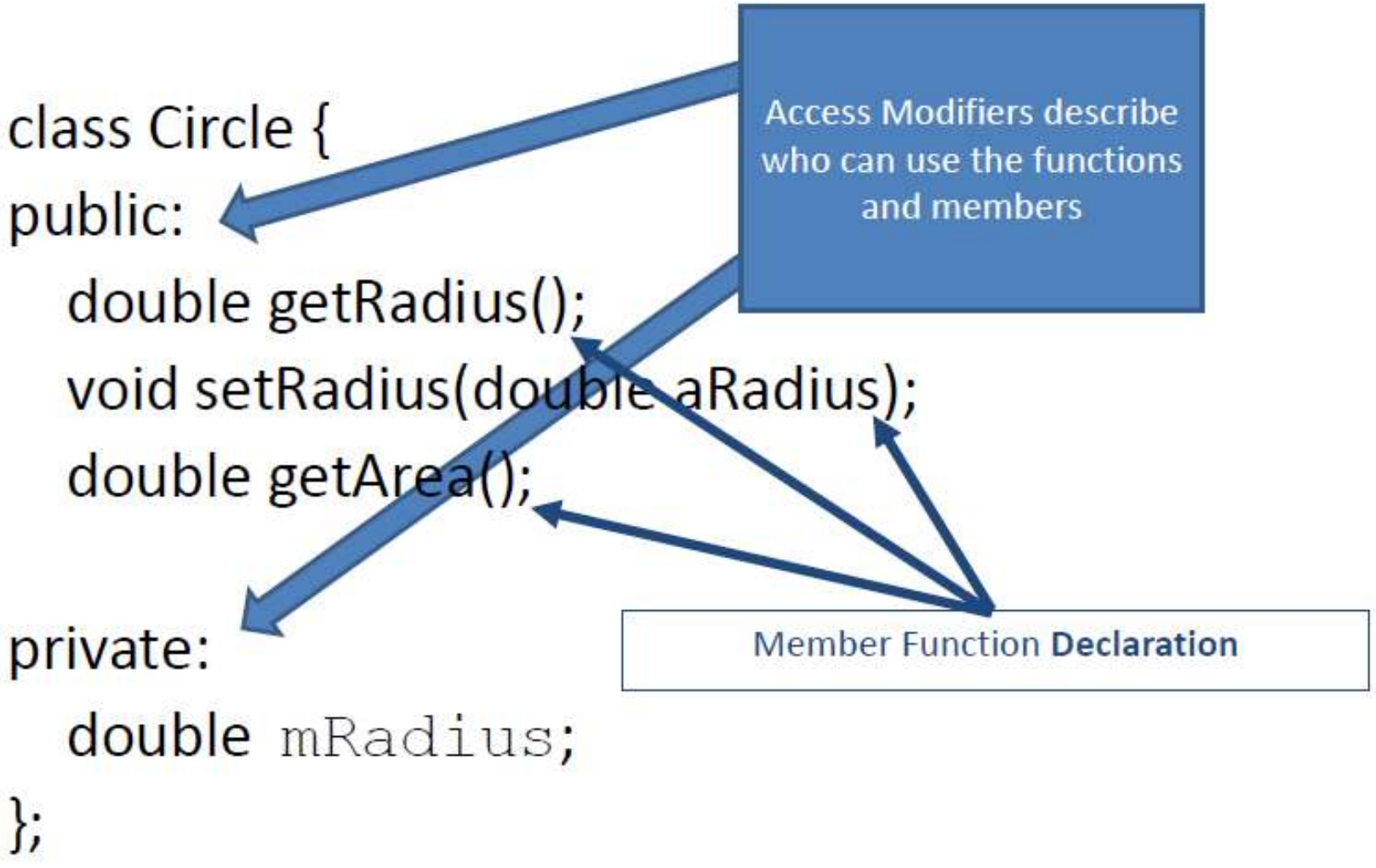
– **Decide on the values to represent**
  - **Data member** radius
– **Decide on the member functions needed**
  - setRadius()
  - getRadius()
  - getArea()

Access Modifiers

- **An access modifier determines which class methods can see and use a member variable or method within a class.**

Access	public	protected	private
Same class	yes	yes	yes
Derived classes	yes	yes	no
Outside classes	yes	no	no

Class Circle Declaration



Defining a Member Function

- **Member functions are declared in the class declaration**
- **If the class is declared and defined in the same file as main, the class declaration should be above the main and the definitions (described here) should be below**

```
void Circle::setRadius(double aRadius)
{ mRadius= aRadius; }
double Circle::getRadius()
{return mRadius;}
double Circle::getArea()
{
     return mRadius*mRadius*PIE;
}
```

Member Function Definition

- **Member function definition syntax:**

Return TypeClassName::functionName (Parameters)
{
Function Body Statement(s);
}

Example:

```
double Circle::getRadius()
{
 return mRadius;
}
```

The '::' Operator

- **'::' is the scope resolution operator**

– **Tells the class a member function is a member of the class.**

– `double Circle::getRadius()` **indicates that function** getRadius is a member of the class "**Circle**"

– The class name that precedes '::' is a class name where the member function belongs to.

The Period '.'

'.' used with variables(object) to call a member function:

Example:

```
Circle circle1,circle2;
 circle1.setRadius(10);
```

Calling Member Functions

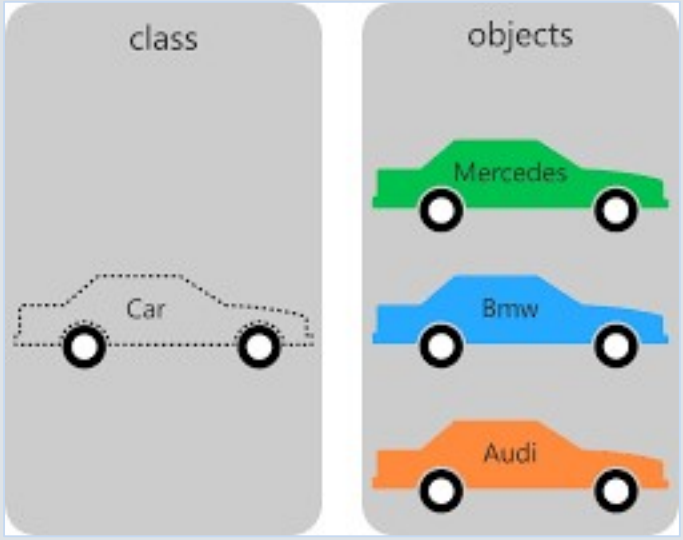
- **Calling the Circle member function output is done in this way:**

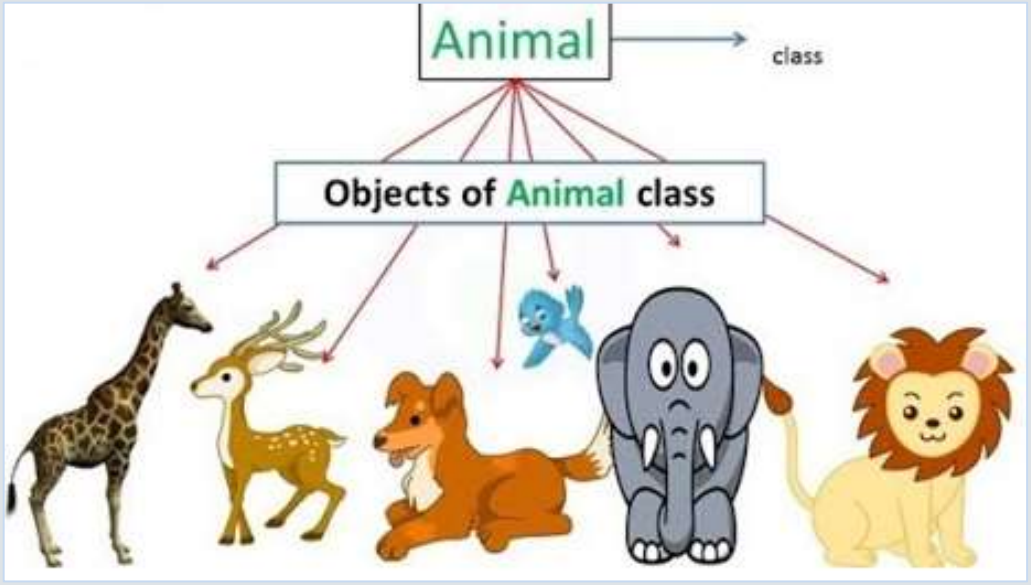
```
cout<< circle1.getRadius() << endl;
cout<< circle2.getRadius() << endl;
```

– Note that **circle1** and **circle2** have their own copies of the radius variable for use by the getRadius() function

Instances

- **Each *instance (object)* has its *own radius***
- **In other words, *each object has its own data member(s)***
- Instances share the same implementation of member functions.





Encapsulation

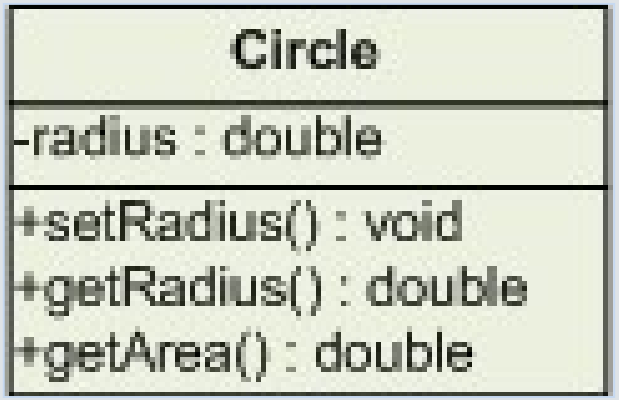
- **Encapsulation is:**

– **Combining** a number of items, such as variables and functions, into a single package such as an object of a class.

- UML Diagram

 - denotes a private member

 + denotes a public member



Public Or Private?

- **C++ helps us restrict the program from directly accessing private member variables.**

– private members of a class can only be accessed within the definitions of member functions.

- If the program tries to access a private member, the compiler gives an error message

– Private members can be variables or functions

Private Variables

- Private variables cannot be accessed directly by the instances (objects).

– Changing their values requires the use of public member functions of the class.

- To set the private radius variable in a Circle class use a member function:

```
void Circle::setRadius(double aRadius)
{
    mRadius= aRadius;
}
```

Public or Private Members

- The keyword private identifies the members of a class that can be accessed only by member functions of the class.
  - Members that follow the keyword private are private members of the class.

- The keyword public identifies the members of a class that can be accessed from outside the class.
  - Members that follow the keyword public are public members of the class.

Using Private Variables

- **It is normal to make all data members private.**
- **We have two types of member functions:**

– *Accessor(getter)* functions allow you to obtain the values of member variables.

Example: getRadius() in class Circle.

– *Mutator(setter)* functions allow you to change the values of member variables.

Example: setRadius() in class Circle

A Bike
(version 1)

– Uses all public data members.

– Notice my convention is to capitalize the first letter of the type name Bike.

```
class Bike {
   public:
         char Name[80];
         int Size;
         double WheelDiameter;
};
```

A Bike

(version 1)

– Simple.

– Suppose we don't find the string *flexible enough* and want to change the name variable to a String.
– This causes a problem for the main function and any users that are utilizing the Bike class.
– The main must be changed.

Bike (version 2)

- Using member functions (accessors and mutators).
- Restrict member variables to private access.
- Using string instead of char array.

```
class Bike {
public:
   string getName();
   void setName(string aName);
   int getSize();
   void setSize(int aSize);
   double getWheelDiameter();
   void setWheelDiameter(double aDiameter);
private:
   string Name;
   int Size;
   double WheelDiameter;
};
```

Bike (version 2)

Member functions definition:

```
string Bike::getName() {return Name;}
void Bike::setName(string aName) {Name= aName;}
int Bike::getSize(){return Size;}
void Bike::setSize(int aSize){Size=aSize;}
double Bike:: getWheelDiameter()
{
     return Diameter;
}
void Bike::setWheelDiameter(double aDiameter)
{
     Diameter=aDiameter;
}
```

Declaring an Object

- **Once a class is defined, an object of the class is declared just as variables of any other type**

– **Example: to create two objects of type Bicycle:**

```
class Bike{
  // class definition lines
  };
  Bike mBike;
  Bike rBike;
```

The Assignment Operator

- **Objects and structures can be assigned values with the assignment operator (=)**
- **Example:**

```
Bike mBike;
 mBike.setName("Foes");
 mBike.setSize(18);
 mBike.setWheelDiameter(26.0);
 Bike mBike2 = mBike;
```

Calling Public Members

- Recall that if calling a member function from the main function of a program, you must include the object

  name:

```
mBike.getName();
```

Calling Private Members

- **When a member function calls a private member function,. an object name is not used.**

The private member function is called by a public one.

```
class Bike {
public:
        string getName();
        void setName(string aName);
        int getSize();
        void setSize(int aSize);
        double getWheelDiameter();
        void setWheelDiameter(double aDiameter);
private:
        bool validSize(int aSize);
        string Name;
        int Size;
        double WheelDiameter;
};
```

Calling Private Members

- **When a member function calls a private member function, an object name is not used:**

void Bike::setSize(int aSize) {

```
if(validSize (aSize))
      Size= aSize;
else
      Size= 10;
}
```

bool Bike::validSize(int aSize) {

```
if(aSize<= 0)   return false;
else            return true;
}
```

example

```
class Carpet
{
 private:
        int length;
        int width;
        double price;
        void setPrice();
 public:
        int getLength();
        int getWidth();
        double getPrice();
        void setLength(int);
        void setWidth(int);
};
```

```
// implementation section
int Carpet::getLength()  {    return length; }
int Carpet::getWidth()   {    return width;  }
double Carpet::getPrice(){    return price;  }
void Carpet::setLength(int len)
{
      length = len;      setPrice();
}
void Carpet::setWidth(int wid)
{
      width = wid;       setPrice();
}
void Carpet::setPrice()
{
const int SMALL = 12; const int MED = 24;
const double PRICE1 = 29.99; const double PRICE2 = 59.99;
const double PRICE3 = 89.99;
  int area = length * width;
  if(area <= SMALL) price = PRICE1;
  else if(area <= MED) price = PRICE2;
  else   price = PRICE3;
}
```